Лекція 2. Class design, Encapsulation

План:

1. **Класи і об'єкти. Поля та методи класу. Конструктори класу.**
2. **Перевантаження методів (статичний поліморфізм).**
3. **Інкапсуляція даних. Модифікатори доступу.**
4. **Статичні поля та методи.**

Усі об'єктно-орієнтовані мови програмування забезпечують реалізацію базових концепцій: *інкапсуляції*, *поліморфізму* та *наслідування*.

Інкапсуляція – це механізм, який зв'язує код і дані, захищаючи у цьому випадку їх від зовнішнього пошкодження і некоректного використання.

Наслідування – це процес створення нового класу шляхом розширення можливостей старого класу через додавання нових властивостей та успадкування старих.

Поліморфізм – це концепція, відповідно до якої використовується спільний інтерфейс для обробки даних різних спеціалізованих типів.

Спільна дія поліморфізму, інкапсуляції та наслідування:

- Найповніше ілюструє силу об'єктно-орієнтованого підходу приклад з реального життя – об'єкт «автомобіль».
- Усі водії використовують наслідування для управління різними типами засобів руху: автобусом, вантажним автомобілем і та іншими засобами пересування.
- Постійно водії зустрічаються з інкапсульованими засобами в автомобілі. Педалі гальма, акселератор, кермо – це частини складного механізму, які пропонують простий інтерфейс, приховуючи при цьому конкретну реалізацію дій.
- Нарешті, поліморфізм, демонструє здатність виробників автомобілів пропонувати все нові варіанти одного і того ж за змістом транспортного засобу. Адже в усіх автомобілях необхідно натиснути на педаль гальма, щоб зупинитись, повернути кермо для зміни напряму руху і т. д.

Центральним елементом об'єктно-орієнтованого програмування є *абстракція*. Завдяки можливості абстрагуватися людство справляється з проявами складних речей в оточуючому світі. Описуючи поведінку якого-небудь об'єкта, наприклад автомобіля, ми будуємо його модель. Модель, як правило, не може описати об'єкт повністю, реальні об'єкти досить складні. Доводиться відбирати тільки ті характеристики об'єкта, котрі важливі для розвязання поставленої перед нами задачі. Для описання вантажоперевезень важливою характеристикою буде вантажопід'ємність автомобіля, а для описання автомобільних гонок вона не суттєва. Але для моделювання гонок обов'язково треба описати метод набору швидкості даним автомобілем, а для вантажоперевезення це не суть важливо. Ми повинні від деяких деталей об'єкта абстрагуватися. Дуже важливо вибрати правильну степінь абстракції. Занадто висока степінь абстракції дасть тільки приблизний опис обєкта, що не дозволить првильно змоделювати його поведінку. Занадто низька степінь зробить модель дуже складною, перевантажену деталями і тому непридатною. Опис кожної

моделі робиться у вигляді одного або декількох **класів** (classes). Клас можна вважати проектом, зліпком, кресленням, по якому потім будуть створюватися конкретні об’єкти.

Клас — це креслення для створення **об'єктів**. Клас — шаблон поведінки об'єктів певного типу з певними параметрами, що визначають стан. Усі примірники одного класу (об'єкти, породжені від одного класу):

● Мають один і той же набір властивостей

● Загальну поведінку, однаково реагують на однакові повідомлення

Послідовність дій при роботі з класами:

1. Описати клас.
2. Реалізувати клас.
3. Створити екземпляр класу - об‘єкт.
4. Працювати з об'єктом, звертаючись до його властивостей і методів.
5. Знищити об'єкт (в Java - автоматично)

Клас є ***шаблоном*** (template) ***об’єкта***, а ***об’єкт*** – ***екземпляром*** (instance) класу.

При створенні класу визначаються дані і код. Зокрема, дані називають ***змінними*** класу, а код, який виконується над цими даними – ***методами*** класу.

Доступ до даних і коду контролюється ***інтерфейсом***.

Синтаксис оголошення класу у мові Java:

```
class ім`я_класу {
тип об’єктна_змінна_1;
тип об’єктна_змінна_2;
. . .
тип об’єктна_змінна_N;
тип ім’я_методу_1(список параметрів) {
//тіло методу
}
тип ім’я_методу_2(список параметрів) {
// тіло методу
}
. . .
тип ім`я_методу_N(список параметрів) {
// тілу методу
}
}
```

Приклад класу:

```
public class Human {
public static void main (String[] args) {
}
/**
 * добавляємо х-ки людини
 */
        public String name; // ім'я
        public int height; //зріст
        public  int weight; //вага
        public  int age; //вік
        /**
         * добавляємо дію людини
         */
        public void go(){ //може ходити
        System.out.println("Я іду додому."+"  А звати мене " + name);
        }
        public String speak() { // може говорити
        return "Привіт! Я" + name;
        }
        public void eat() {//може їсти
        System.out.println("Я іду");
        }
}|
```

Методи і змінні, визначені всередині класу, називають ***членами*** (members) цього класу.

Дані або змінні, визначені у блоці class, можуть належати конкретному екземплярові класу (об'єктові), внаслідок чого їх називають ***змінними екземпляра***, або бути глобальними змінними (instance variables), спільними для всіх екземплярів конкретного класу і носити назву ***змінних класу***.

Для визначення змінної класу перед її оголошенням необхідно додати модифікатор **static**.

Якщо до оголошення змінної додати модифікатор final, то значення змінної не можна змінювати в підкласах.

Створення об'єктів:

Той факт, що в програмі описаний клас, ще не означає, що ми можемо працювати з його властивостями і викликати його методи.

Для роботи потрібно:

1) оголосити змінну типу Клас
2) створити об'єкт (екземпляр) класу за допом. **new**

Синтаксис:

ІмяКласу імяОб'єкта; // Оголошення змінної-об'єкта
ІмяОб'єкта = new ІмяКласу ([параметри конструктора]);

Можна поєднати оголошення і створення об'єкта:

ІмяКласу імяОб'єкта = new ІмяКласу ([параметри конструктора]);

Програмний код міститься в ***методах***. Головні частини методу – це ***ім'я***, ***параметри***, тип ***значення***, яке повертається, і ***тіло***.

Для виходу з методу використовується оператор **return**, за яким іде вираз, значення якого необхідно повернути.

Метод може повертати значення довільного типу або не повертати нічого. У таких випадках на місці типу в оголошенні методу необхідно написати слово **void**.

Тип ім'я_методу (список параметрів) {
//тіло методу
}

Для створення ***об'єкту*** (екземпляру класу), використовується оператор **new**. Оператор new динамічно (під час виконання) розміщує об'єкт в оперативній пам'яті загального призначення.

Ім'я_Класу змінна_класу = new Ім'я_Класу();

Оператор new виокремлює пам'ять для об'єкта під час виконання програми.

Отже, механізм створення об'єктів:

1. Створюється змінна посилального типу (посилання) – комірка в пам'яті комп'ютера для зберігання адреси майбутнього об'єкта.
2. Виділяється простір для розміщення об'єкта з усіма його властивостями (атрибутами).
3. Атрибути ініціалізовані значеннями по замовчуванню.
4. Виконується явна ініціалізація атрибутів, якщо вона була задана програмістом.
5. Виконується конструктор.
6. Посилання на створений об'єкт (його адресу) записується в відповідну змінну посилального типу.

Використання об'єкта здійснюється за допомогою доступу до його елементів.

Для доступу до елементів використовується оператор «крапка» після змінної-посилання на об'єкт:

імяОб'єкта.імяВластивості = значення;

імяОб'єкта.імяМетода (параметри);

Наприклад:

Human H = new Human();
H.name = "Pupkin";
H.go();

Інкапсуляція:

- доступ до стану об'єкта напряму заборонено
- ззовні з ним можна взаємодіяти виключно через заданий інтерфейс (відкриті поля та методи), що дозволяє знизити зв'язність.
- контролюються звернення до полів класів та їхня правильна ініціалізація, усуваються можливі помилки пов'язані з неправильним викликом методу.
- розробники класу можуть як-завгодно змінювати всі закриті елементи і навіть перейменовувати та видаляти їх, не турбуючись, що десь хтось їх використовує у своїх програмах.
- рекомендується з самого початку створювати закриті поля і лише в разі крайньої необхідності надавати ширший доступ до них. Для роботи із закритими полями класів краще використовувати відповідний метод доступу (getter) та метод мутатор (setter).
- *Get-методи*. Викликаються з метою зчитування значення змінної класу (взяти внутрішню змінну).

- *Set-методи*. Викликаються з метою присвоєння значення змінній класу (присвоїти внутрішній змінній нове значення).

  Приклад:

```
5    * @author Tim
6    */
7    public class Human {
8
9        private int height; // рост
10       private int age; // возраст
11       private int weight; // вес
12       private String name;// имя
13
14       public int getHeight(){
15           return height;
16       }
17
18       public void setHeight(int height){
19           if (height<0) {
20               System.out.println("Height не может быть меньше нуля");
21               return;
22           }
23           this.height = height;
24       }
```

public String name; // ім'я
public int height; //зріст
public int weight; //вага

```
1    package hellojava;
2
3    public class Main {
4
5        public static void main(String[] args) {
6            Human h = new Human();
7            h.setHeight(160);
8            h.eat();
9            h.hunt();
10           h.go();
11       }
```

- Якщо доступ до власних членів класу збоку інших частин програми можна отримати тільки через загальнодоступні методи, то можна вважати, що **принцип інкапсуляції цілковито реалізовано.**

Класифікація методів

1. ***Конструктори***. Викликаються для створення екземплярів об'єктів визначеного класу.

2. ***Деструктори***. Викликаються, коли робота з екземпляром об'єкта закінчена.

3. ***Копіювальники***. Використовуються для копіювання екземпляра об'єкта в інший об'єкт.

4. ***Set-методи***. Викликаються з метою присвоєння значення змінній класу.

5. ***Get-методи***. Викликаються з метою зчитування значення змінної класу.

6. Методи ***введення-виведення***. Викликаються для виконання взаємодії із зовнішніми пристроями.

7. Методи, специфічні для області визначення застосування.

Способи передачі аргументів у методи

Прості типи в мові Java передаються за ***значеннями*** (call-by-value), тобто у формальний параметр копіюється значення аргументу Все, що відбувається з параметром, не впливає на аргумент.

Об'єкти передаються за ***посиланням*** (call-by-reference). Тобто параметрові присвоюється значення посилання на об'єкт. Обидва посилання – і аргумент, і параметр – вказують на один і той же об'єкт у пам'яті. Тому всі зміни, які відбулися в методі, впливають на об'єкт, переданий як аргумент.

Визначення конструктора у мові Java

Конструктор (constructor) – здійснює ініціалізацію об'єкта при його створенні. Ім'я конструктора завжди збігається з іменем класу, а за синтаксисом він нагадує метод. При написанні конструктора явно не вказується тип значення, яке він повертає, зокрема і void.

Якщо конструктор класу не описано явно, Java створює для цього класу конструктор ***за замовчуванням***. Конструктор за замовчуванням присвоює змінним класу нульові значення.

Конструктор може мати список визначених параметрів, які передаються у нього під час створення об'єкту.

Перевантаження методів і конструктора класу

Одним із способів реалізації поліморфізму є переванта-ження методів та конструктора класу (metod overloa-ding). У мові Java можна в одному класі визначати декі-лька методів з одним іменем, однак різною кількістю параметрів. Ці методи називають ***перевантаженими*** (overloaded).

При перевантаженні методу, щоб визначити, яку версію методу треба виконати, використовується кількість і (або) тип аргументів. Типи значень, які повертає метод, можуть бути різними і вони не використовуються як ознака, за якою розрізняють дві версії методу.

Керування доступом у мові Java

Доступ до елементів класу задається за допомогою ***специфікаторів доступу*** (access control).

У мові Java існують наступні специфікатори доступу: public, private та protected.

private – члени класу доступні тільки всередині класу;

default (модифікатор, по замовчуванню) – члени класу доступні всередині пакету;

protected – члени класу доступні всередині пакету і під час наслідування;

public – члени клас доступні для всіх класів.

Статичні поля, методи та блоки

Для створення статичного елементу класу необхідно перед його оголошенням поставити ключове слово static. До таких членів класу можна звертатися ще до створення хоча б одного екземпляра (об'єкта) класу. Статичними можуть бути як змінні класу, так і методи класу.

Екземпляри змінних, оголошених як static є за своїм змістом є глобальними. При створенні об'єктів даного класу змінні не копіюються. Усі екземпляри класу використовують одну і ту ж статичну змінну.

Через статичні елементи класу в мові Java реалізовано аналоги глобальних функцій і глобальних змінних.

На методи, оголошені як static накладено наступні обмеження:

1) вони викликають тільки статичні методи;

2) вони використовують тільки статичні змінні;

3) вони не можуть посилатися на this і super.

Виклик статичного методу має наступну форму:

Ім’я_Класу.ім’я_Методу();

В Java можна оголосити блок як static, який виконається тільки один раз – під час першого завантаження класу.

Типи відносин між класами

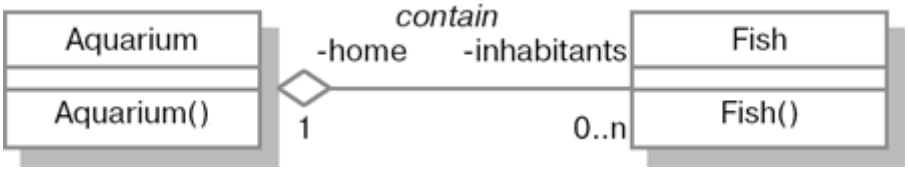
Як правило, будь-яка програма, написана на об'єктно-орієнтованій мові, є певним набором пов'язаних між собою класів. Можна провести аналогію між написанням програми і будівництвом будинку. Подібно до того, як стіна складається з цегли, комп'ютерна програма з використанням ООП будується з класів. Причому ці класи повинні мати уявлення один про одного, для того щоб спільно виконувати поставлене завдання.

Можливі такі зв'язки між класами в рамках об'єктної моделі (наводяться лише найбільш прості і часто використовувані види зв'язків, докладний їх розгляд виходить за рамки цієї ознайомчої лекції):

- агрегація (Aggregation);
- асоціація (Association);
- успадкування (Inheritance);
- метакласи (Metaclass).

Агрегація

Відношення між класами типу "містить" (contain) або "складається з" називається агрегацією, або включенням. Наприклад, якщо акваріум наповнений водою і в ньому плавають рибки, то можна сказати, що акваріум агрегує в собі воду і рибок.



Таке ставлення включення, або агрегації (aggregation), зображується лінією з ромбиком на боці того класу, який виступає в якості власника, або контейнера. Необов'язкова назва відносин записується посередині лінії.

У нашому прикладі ставлення contain є двонаправленим. Об'єкт класу Aquarium містить кілька об'єктів Fish. У той же час кожна рибка "знає", в якому саме акваріумі вона живе. Кожен клас має свою роль в агрегації, яка вказує, яке місце займає клас у даному відношенні. Ім'я ролі не є обов'язковим елементом позначень і може бути відсутнім на діаграмі. У прикладі можна бачити роль home класу Aquarium (акваріум є домом для рибок), а також роль inhabitants класу Fish (рибки є мешканцями акваріума). Назва ролі звичайно збігається з назвою відповідного поля в класі. Зображення такого поля на діаграмі зайве, якщо вже вказано ім'я ролі. Тобто в даному випадку клас Aquarium матиме властивість (поле) inhabitants, а клас Fish - властивість home.

Кількість об'єктів, які беруть участь у відношенні, записується поряд з ім'ям ролі. Запис "0 .. n" означає "від нуля до нескінченності". Прийнято також позначення:

"1 .. n" - від одиниці до нескінченності;
"0" - нуль;
"1" - один;
"N" - фіксована кількість;
"0 .. 1" - нуль чи один.

Код, що описує розглянуту модель і явище агрегації, може виглядати, наприклад, наступним чином:

```
// Визначення класу Fish
public class Fish {
  // Визначення поля home
  // (Посилання на об'єкт Aquarium)
  private Aquarium home;
  public Fish () {
  }
}
// Визначення класу Aquarium
public class Aquarium {
  // Визначення поля inhabitants
  // (Масив посилань на об'єкти Fish)
  private Fish inhabitants [];
  public Aquarium () {
  }
}
```

Асоціація

Якщо об'єкти одного класу посилаються на один або більше об'єктів іншого класу, але ні в ту, ні в іншу сторону відношення між об'єктами не носить характеру "володіння", або контейнеризації, таке ставлення називають асоціацією (association). Відношення асоціації зображується так само, як і ставлення агрегації, але лінія, що зв'язує класи, - проста, без ромбика.

Як приклад можна розглянути програміста і його комп'ютер. Між цими двома об'єктами немає агрегації, але існує чіткий взаємозв'язок. Так, завжди можна встановити, за якими комп'ютерами працює будь-якої програміст, а також які люди користуються окремо взятим комп'ютером. У розглянутому прикладі має місце асоціація "багато-до-багатьох".

У даному випадку між екземплярами класів Programmer і Computer в обидві сторони використовується відношення "0 .. n", тому що програміст, в принципі, може не працювати з комп'ютером (якщо він теоретик або на пенсії). У свою чергу, комп'ютер може ніким не використовуватися (якщо він новий і ще не встановлений).

Код, відповідний розглянутому наприклад, буде, наприклад, таким:

```
public class Programmer {
  private Computer computers [];
```

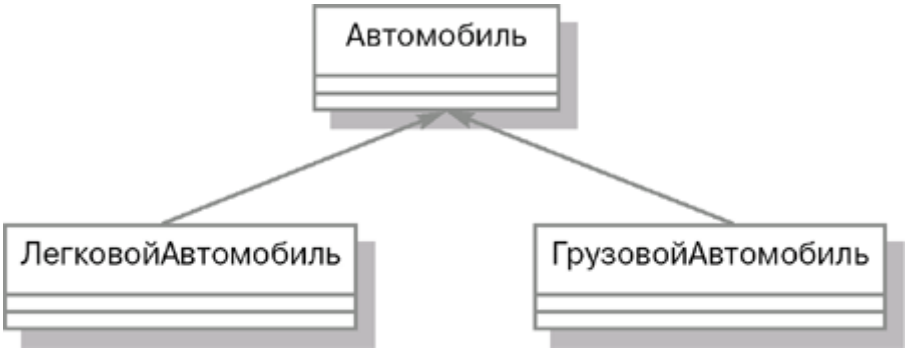
```
  public Programmer () {
  }
}
public class Computer {
  private Programmer programmers [];
  public Computer () {
  }
}
```

Спадкування

Спадкування є важливим випадком відносин між двома або більше класами.

Спадкування (inheritance) - це відношення між класами, при якому клас використовує структуру або поведінку іншого класу (одиночне спадкоємство), або інших (множинне спадкоємство) класів. Спадкування вводить ієрархію "загальне / приватне", в якій підклас успадковує від одного або декількох більш загальних суперкласів. Підкласи зазвичай доповнюють або перевизначають успадковану структуру і поведінку.

Як приклад можна розглянути задачу, в якій необхідно реалізувати класи "Легковий автомобіль" і "Вантажний автомобіль". Очевидно, ці два класи мають загальну функціональність. Так, обидва вони мають 4 колеса, двигун, можуть переміщатися і т.д. Всіма цими властивостями володіє будь-який автомобіль, незалежно від того, вантажний він чи легковий, 5 - або 12-місний. Розумно винести ці загальні властивості і функціональність в окремий клас, наприклад, "Автомобіль" і успадковувати від нього класи "Легковий автомобіль" і "Вантажний автомобіль", щоб уникнути повторного написання одного й того ж коду в різних класах.



Відношення узагальнення позначається суцільною лінією з трикутною стрілкою на кінці. Стрілка вказує на більш загальний клас (клас-предок або суперклас), а її відсутність - на більш спеціальний клас (клас-нащадок або підклас).

Використання наслідування сприяє зменшенню кількості коду, створеного для опису схожих сутностей, а також сприяє написанню більш ефективного та гнучкого коду.

У розглянутому прикладі застосовано одиночне спадкування. Деякий клас також може успадковувати властивості та поведінку відразу декількох класів.

Найбільш популярним прикладом застосування множинного спадкоємства є проектування системи обліку товарів у зоомагазині.

Всі тварини в зоомагазині є спадкоємцями класу "Тварина", а також спадкоємцями класу "Товар". Тобто всі вони мають вік, мають потребу в їжі і воді і в той же час мають ціну і можуть бути продані.

Множинне спадкування на діаграмі зображується точно так само, як одиночне, за винятком того, що лінії спадкоємства сполучають клас-нащадок відразу з декількома суперкласами.

Не всі об'єктно-орієнтовані мови програмування містять мовні конструкції для опису множинного спадкоємства.

У мові Java множинне успадкування має обмежену підтримку через інтерфейси.

Метакласи

Отже, будь-який об'єкт має структуру, що складається з полів і методів. Об'єкти, що мають однакову структуру і семантику, описуються одним класом, який і є, по суті, визначенням структури об'єктів, породжених від нього.

У свою чергу, кожен клас, або опис, завжди має строгий шаблон, що задається мовою програмування або вибраною об'єктною моделлю. Він визначає, наприклад, чи допустиме множинне спадкування, які існують обмеження на іменування класів, як описуються поля і методи, набір існуючих типів даних та багато іншого. Таким чином, клас можна розглядати як об'єкт, у якого є властивості (ім'я, список полів та їх типи, список методів, список аргументів для кожного методу і т.д.). Також клас може мати певну поведінку, тобто підтримувати методи. А раз для будь-якого об'єкту існує шаблон, що описує властивості і поведінку цього об'єкта, значить, його можна визначити і для класу. Такий шаблон, що задає різні класи, називається метакласом.

Щоб уявити собі, що таке метакласи, розглянемо приклад якоїсь бюрократичної організації. Будемо вважати, що всі класи в такій системі мають суворі інструкції, які описують, що потрібно зробити, щоб породити новий об'єкт (наприклад, найняти нового службовця або відкрити новий відділ). Як і годиться класам, вони описують всі властивості нових об'єктів (наприклад, зарплату і професійний рівень для співробітників, площа та майно для відділів) та їх поведінка (обов'язки службовців і функції підрозділів).

У свою чергу, написання нової інструкції можна строго регламентувати. Скажімо, необхідно використовувати спеціальний бланк, дотримуватися правил оформлення і заповнити всі обов'язкові поля (наприклад, номер інструкції та прізвища відповідальних працівників). Така "інструкція інструкцій" і буде представляти собою метаклас в ООП.

Отже, об'єкти породжуються від класів, а класи - від метакласів. Він, як правило, в системі тільки один. Але існують мови програмування, в яких можна створювати і використовувати власні метакласи, наприклад мова Python. Зокрема, функціональність метакласів може бути наступна: при формуванні класу він буде переглядати список усіх методів в класі і, якщо ім'я методу має

вид set_XXX або get_XXX, автоматично створювати поле з ім'ям XXX, якщо такого не існує.

Оскільки метаклас сам є класом, то немає ніякого сенсу у створенні "мета-мета-класів".

У мові Java також є метакласи. Це клас, який так і називається - Class (описує класи), він розташовується в основній бібліотеці java.lang. Віртуальна машина використовує його за прямим призначенням. Коли завантажується черговий. Class-файл, що містить опис нового класу, JVM породжує об'єкт класу Class, який буде зберігати його структуру. Таким чином, Java використовує концепцію метаклассов в самих практичних цілях. За допомогою Class реалізована підтримка статичних (static) полів і методів. Нарешті, цей клас містить ряд методів, корисних для розробників. Вони будуть розглянуті в наступних лекціях.

Знищення об‘єкта:

- У Java немає деструкторів класу
- Знищення невикористовуваних об'єктів здійснюється автоматично «збирачем сміття» (garbage collector), спеціальним механізмом JVM
- Об'єкт видаляється, коли в подальшій програмі на нього немає жодного звернення

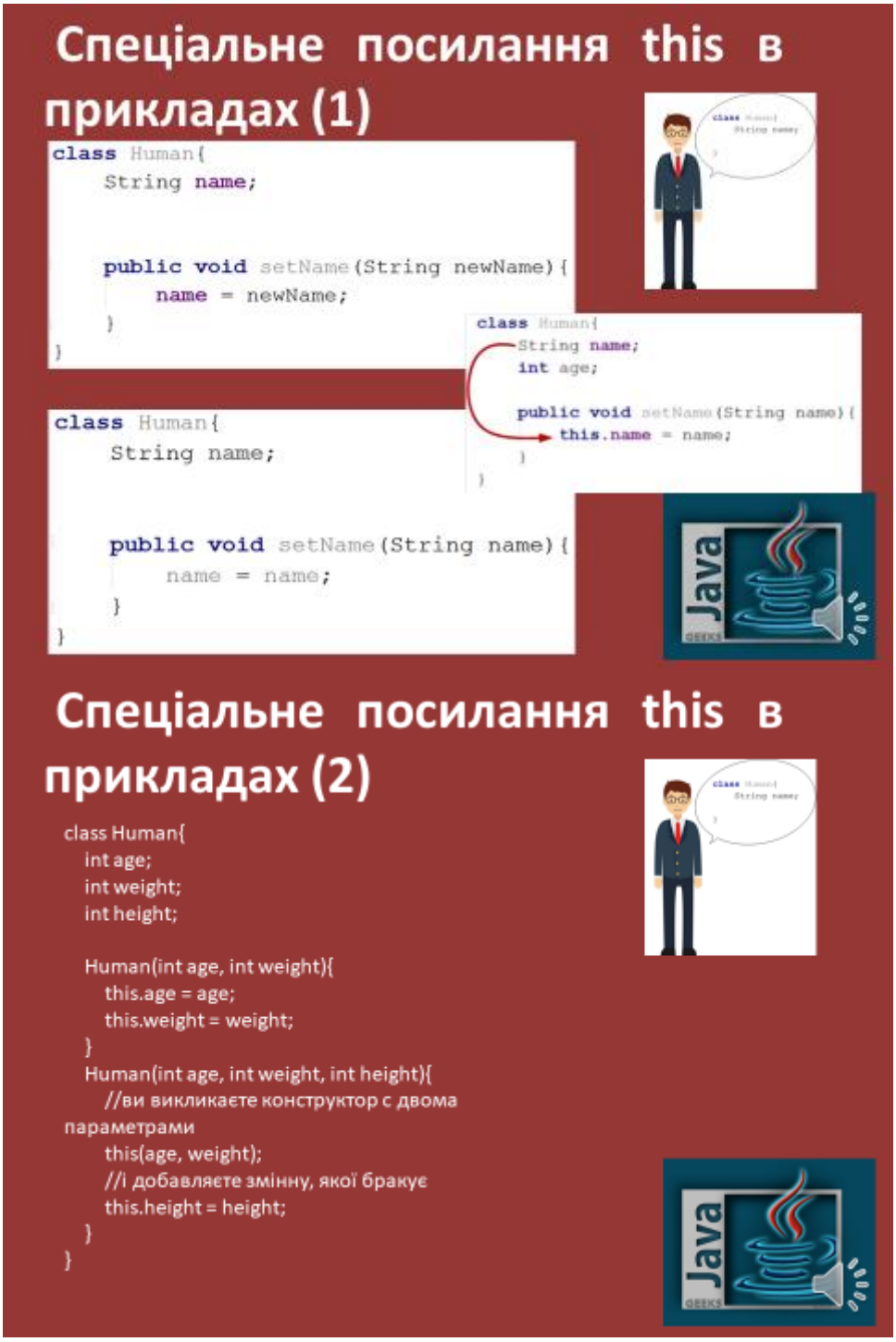
Спеціальне посилання this:

this - це посилання на поточний екземпляр об‘єкта.

Може застосовуватися тільки в методах класу.

За допомогою this можна:

- звертатися до атрибутів поточного об'єкта
- викликати методи поточного об'єкта
- передавати посилання на поточний об'єкт методам іншого класу



Робота з файлом:

1. Коли Ви створите клас, створиться файл з розширенням **.java.** Назва файла грає дуже важливу роль в Java. Компілятор Java вимагає, щоб ми використовували розширення Java. В Java код повинен постійно знаходитися в класі, а отже наше

ім’я класу і ім’я файла **повинні співпадати**.

Java чутливий до регістру. Тому потрібно впевнитись, що ім’я файла співпадає з ім’ям класу.

2. Клас розміщується в пакеті. Пакет – спосіб логічної організації коду класів у єдину групу. Пакету відповідає каталог файлової системи. Пакет

задається за допомогою ключового слова package. Звертання до класу пакета: mypackage.MyClass.

3. Між пакетом та класом може розміщуватися напис

import java.io.*.

Команда імпорту в програмі говорить компілятору, де шукати файл класу, який ми використовуємо в нашому коді. Він включає визначення на клас, в якому ми потребуємо для виконання коду, io - пакет, який завжди знаходиться в папці на ім'я Java, і цей пакет у свою чергу має номери класів, які мають безліч методів. Ці методи можуть використовуватися в нашому коді, якщо ми імпортуємо пакет.

4. **public class *Humen{}***

Цей рядок об'являє клас з іменем класа/файла. Зверніть увагу,що ім'я класа починається з великої букви. Опис класу знаходиться в рамках фігурних дужок.

5. Автоматично створюється метод ***main*** – це точка входу в програму і при запуску віртуальна машина Java буде шукати цей метод, щоб почати програму.

```
public class Main {

    public static void main(String[] args) {
        Human H = new Human();
        H.name = "Vika";
        H.go();
    }}
```

- main метод оголошений як public так, щоб JVM могла звернутися до цього методу.
- Ключове слово **static** дозволяє методу main () викликатися без потреби створювати екземпляр класу.
- Клас, який не має основного методу, може бути успішно відкомпільований, але не може бути виконаний, оскільки він не має відправної точки виконання, якою є main () метод.

Однак у мові Java дозволено використовувати кілька методів з назвою main() навіть в одному класі. Тому по-справжньому головний метод містить аргументи командного рядка (String [] args).

6. Всі інструкції в Java закінчуються крапкою з комою (;)

Висновки: на лекції ознайомилися з інкапсуляцією – одним із базових концепцій ООП, класом та екземпляром класу, конструктором для побудови екземплярів, перевантаженням конструкторів. Навчились працювати з файлом.

Контрольні запитання:

1. Що таке клас?
2. Що таке екземпляр класу?
3. Що таке конструктор, та скільки їх може бути в програмі?
4. Що таке інкапсуляція? Для чого вона потрібна в програмах?
5. Що таке сетери і гетери, як вони працюють?
6. Які є модифікатори доступу?